

Informe Técnico de Integración de IA

Proyecto "El Tenedor"

NOMBRES: Antonia Yelincic Álvarez, Paula Andrea Castillo Parra, María José Silva

CARRERA: Analista Programador

ASIGNATURA: Programación Back End

PROFESOR: Paulo Simunovic

FECHA: 07-10-2026



1 Descripción de la aplicación

El Tenedor es el punto de venta de un local gastronómico en Punta Arenas. El backend es una API REST en Django y Django REST Framework. El personal de caja usa una PWA con modo offline; el cliente arma su pedido en la web y paga con Webpay. La vista publicada está en <https://el-tenedor.mrjslvsnchz.workers.dev/> (caja) y <https://el-tenedor.mrjslvsnchz.workers.dev/pedir> (pedidos). Repositorio del proyecto: <https://github.com/MariaJoseSS20/el-tenedor>

Modelos y relaciones principales:

- CustomUser: usuario con rol administrador o cajero (1:N con Venta y CajaDiaria).
- Producto 1:N DetalleVenta y 1:N DetallePedido.
- Venta N:1 cajero, 1:N DetalleVenta, 1:N PagoVenta y 1:1 Pedido web.
- Pedido N:1 ZonaDelivery y 1:N DetallePedido.
- Inventario (empaques) y HorarioPedidosWeb (configuración única del horario online).

La API versionada vive en /api/v1/ (productos, ventas, inventario, pedidos, zonas-delivery, caja-diaria). /api/ responde igual, porque la PWA ya publicada usa ese prefijo. El entorno local se crea con python3 -m venv .venv; las dependencias están en requirements.txt y las variables, sin claves reales, en .env.example.

2 Bitácora de IA (tres consultas)

Herramienta: Cursor. Abajo va la transcripción de los prompts y lo que se decidió hacer con cada respuesta. El detalle tabla a tabla está en SECURITY_PROMPTS.md.

Consulta 1. Autenticación JWT y fuerza bruta

Prompt: "Actúa como experto en backend. ¿Qué prácticas de seguridad debo aplicar en Django REST Framework para autenticación con JWT y protección de endpoints? Incluye expiración de tokens y throttling." La IA recomendó SimpleJWT (access + refresh), IsAuthenticated por defecto, permisos por rol y un límite de intentos en el login.

Consulta 2. Verbos HTTP, códigos de estado y recursos

Prompt: "Revisa la estructura de mis rutas y respuestas JSON. ¿Qué cambios sugieres para alinearlas con la arquitectura RESTful? Considera verbos HTTP, códigos de estado y recursos del POS El Tenedor." La IA recomendó ViewSets, rutas con nombre de recurso, 201 al crear, 400 si el JSON no valida, 401/403 si falla la auth, y no borrar ventas.

Consulta 3. CORS, secretos y sanitización

Prompt: "Audita esta API Django REST en tres puntos: restricción de CORS, almacenamiento de secretos en variables de entorno y sanitización de entradas. ¿Cómo evito inyección SQL, que el cliente fije precios o totales, y que alguien se registre como administrador?" La IA recomendó lista blanca de orígenes, secretos solo en el entorno, ORM sin SQL crudo, validación en el serializer y registro limitado al rol cajero.

3 Análisis crítico

No se copió cada sugerencia al pie de la letra. Se evaluó si encajaba con un local que cobra durante un turno y debe conservar la auditoría de caja.

De la consulta 1 se implementó JWT (POST /api/v1/token/ y /api/v1/token/refresh/), permiso IsAuthenticated por defecto y throttling: 10 peticiones por minuto en login, registro y refresh; 60/min anónimo y 120/min autenticado en el resto. El access token dura 8 horas, no 5 o 15 minutos. Una expiración más corta obligaría a la cajera a reingresar en medio del servicio. El refresh dura 7 días y no rota, para que el modo offline pueda reintentar la sincronización. Los permisos de pos/permissions.py separan cajero (vende y consulta) de administrador (catálogo, anular, caja y reportes).

De la consulta 2 se pasaron productos, inventario, zonas y ventas a ViewSets, con 201, 400, 401, 403 y 405 según el caso. Se rechazó un DELETE real sobre ventas: borrar una boleta rompe el cierre de caja. DELETE en /api/v1/ventas/{id}/ responde 405. La anulación es PATCH {"estado":"anulada"} y solo la hace el administrador. Productos, inventario y zonas sí permiten DELETE, porque no son documentos contables. El prefijo /api/v1/ se agregó como pide la pauta; /api/ se dejó activo para no romper la PWA publicada.

De la consulta 3 se dejó CORS_ALLOWED_ORIGINS en el entorno y CORS_ALLOW_ALL_ORIGINS solo cuando DEBUG es verdadero. SECRET_KEY sale del entorno; si DEBUG=False y sigue la clave de desarrollo,



Django no arranca. `.env.example` lista las variables sin claves reales. No hay SQL crudo: todo pasa por el ORM, que parametriza las consultas. Los serializers rechazan precio menor o igual a 0, cantidad menor que 1 y ventas sin detalle. El total, el cajero y la fecha los fija el servidor, aunque el JSON del cliente traiga otros valores. `POST /api/v1/registro/` crea siempre un cajero; el rol administrador no se acepta en el body.

En resumen, la IA sirvió para revisar autenticación, forma REST y exposición de secretos. Se aplicó lo que reduce riesgo (JWT, roles, throttle, CORS, `.env`, validación) y se descartó lo que chocaba con la operación del local (access token muy corto y borrado físico de ventas).



APRENDER HACIENDO A LO LARGO DE LA VIDA



INSTITUTO PROFESIONAL INACAP

ACREDITADO • NIVEL DE EXCELENCIA

- Gestión Institucional • Docencia de Pregrado
- Vinculación con el Medio
- Próxima Acreditación Septiembre 2030

www.inacap.cl